

ECOCIVIX AI

06 — Technical Architecture

TEAM VELOCITY • HackYatra × EDM 3.0 • PRE-HACKATHON PLANNING

Purpose: Define the planned application architecture for a real 24-hour hackathon MVP while avoiding unnecessary complexity.

Status: PRE-HACKATHON DRAFT / PLANNING SPECIFICATION

Approval authority: Garv Nain

Architecture

- Frontend: React + Vite + JavaScript.
- Backend: Node.js + Express + JavaScript.
- Database/Auth/Storage: Supabase / PostgreSQL / Supabase Auth / Supabase Storage.
- AI: Gemini or another suitable free-tier provider available under the official hackathon constraints.
- Maps: suitable free/open mapping solution where practical.

High-Level Flow

- Citizen/Admin UI → Express API → authentication/authorization → business logic → Supabase database/storage → AI provider where required → validated response → UI.

Frontend Responsibilities

- Routing, forms, state, accessibility, rendering, user feedback, API consumption and responsive UX.
- Frontend must never hold server secrets.

Backend Responsibilities

- Authentication verification, authorization, issue lifecycle, AI orchestration, validation, persistence, assignments, updates, error handling and security boundaries.

Database Responsibilities

- Users/roles, issues, AI analysis, updates, departments, assignments and required metadata.
- Schema is finalized in Document 07.

Security

- Secrets remain server-side. Never commit .env files or service keys. Validate all inputs. Verify authorization server-side. Rotate exposed credentials immediately.

Error Architecture

- Use consistent HTTP status behavior and structured errors.
- Frontend receives safe human-readable messages; internal secrets/stack traces remain server-side.

Observability

- During hackathon use clear server logs for requests, failures, AI provider errors and database failures without logging secrets or sensitive user content unnecessarily.

Deployment

- Deploy frontend and backend to free/low-cost services compatible with the hackathon. Use managed Supabase services for database/auth/storage where appropriate.

Performance

- Keep the core report-to-AI-to-database path lightweight. Lazy-load heavy visuals, optimize images and keep 3D optional.

Architecture Constraints

- Do not add microservices unless necessary. Do not add a second database without a concrete need. Do not add paid infrastructure that creates demo risk.

Acceptance Criteria

- Clear frontend/backend boundary; real database; real authentication; server-side AI; validated AI output; secure configuration; deployable MVP; graceful failure behavior.